

Day 6 Beginner-Friendly Full Slide Content

Course metadata

Course Title: G1 Locomotive and Arm Control Infrastructure **Day:** 6 **Target Audience:** Beginners **Instructor:** Manus AI

Teaching design notes

This deck is designed for beginners, focusing on the practical aspects of G1 humanoid locomotion and arm control. It simplifies complex control theory, emphasizing the interaction between high-level commands and low-level hardware. Each slide includes clear purpose statements, on-slide text, detailed speaker notes, and cues for visual diagrams to enhance understanding.

Section summary table

Section	Focus	Suggested Slide Range
Cover and Learning Map	Day 6 scope, control flow, locomotion vs. manipulation	1–4
Locomotive Architecture & FSM	System layers, state management, transition logic	5–15
Velocity, Balance & JSON-CLI	Velocity commands, stance configuration, CLI structures	16–28
Arm Control & SDK Merging	Joint SDKs, Action APIs, coordination architecture	29–42
Integration & Execution	Synchronized movement, feedback loops, lab validation	43–52
Wrap-Up & Review	Readiness check, Day 6 outcome summary	53–56

Cover

Section: Cover **Purpose:** Introduce the course day and its focus. **On-slide text:**

Day 6: G1 Locomotive and Arm Control Infrastructure

Speaker notes: Welcome to Day 6! Today, we will explore how to make the G1 humanoid move and interact with its environment. We'll cover the core principles of locomotive control, arm manipulation, and how to integrate these systems effectively and safely.

Diagram cue: Vinci AI logo, subtle diagonal watermark, copyright tag.

Slide 1: Learning Map & Control Flow

Section: Cover and Learning Map **Purpose:** Outline the day's learning objectives and introduce the control flow. **On-slide text:**

Learning Map: Navigating G1 Control

Initialize FSM → Set Stance → Command Velocity → Monitor Balance → Control Arms → Sync Execution

Control Discipline: Precision and Predictability

Speaker notes: Our journey today will take us through the architecture of G1's movement, from initiating its state machine to commanding its arms and ensuring synchronized execution. We'll emphasize control discipline – the importance of precise and predictable commands for safe and effective operation.

Diagram cue: Flowchart with arrows illustrating the learning path. A gear icon representing control discipline.

Slide 2: Locomotive Control Subsystems - Introduction

Section: Locomotive Architecture & FSM **Purpose:** Introduce the concept of locomotive control and its subsystems in G1. **On-slide text:**

G1 on the Move: Locomotive Control Subsystems

- **Definition:** The systems responsible for G1's movement and balance.
- **Key Components:** High-level controllers, FSM, low-level motor drivers.

Speaker notes: G1's ability to walk and maintain balance is thanks to its locomotive control subsystems. We'll break down this architecture, understanding how high-level commands translate into physical motion through a series of interconnected components.

Diagram cue: High-level block diagram of G1's locomotive control system, showing the interaction between high-level control, FSM, and motor drivers.

Slide 3: System Architecture for Movement

Section: Locomotive Architecture & FSM **Purpose:** Detail the layered architecture of G1's movement control. **On-slide text:**

Layers of Motion: G1's Movement Architecture

- **High-Level Controller:** Receives user commands (e.g., move forward, turn).
- **Mid-Level FSM:** Manages states like 'Idle', 'Walk', 'Stand'.
- **Low-Level Motor Drivers:** Directly control joint actuators.

Speaker notes: G1's movement is orchestrated through a layered architecture. User commands are interpreted by a high-level controller, which then instructs a mid-level Finite State Machine. The FSM, in turn, coordinates with low-level motor drivers to execute precise movements. This hierarchical approach ensures both flexibility and control.

Diagram cue: Layered diagram showing the flow from high-level commands to low-level motor control, with FSM as an intermediary.

Slide 4: Finite State Machine (FSM) State Management

Section: Locomotive Architecture & FSM **Purpose:** Explain the role of FSM in managing G1's locomotive states. **On-slide text:**

G1's Brain: FSM for State Management

- **States:** Discrete modes of operation (e.g., Idle , Walk , Stand , Sit).
- **Transitions:** Rules for moving between states (e.g., Idle to Walk on 'move' command).

- **Events:** Triggers for state changes (e.g., user input, sensor data).

Speaker notes: The Finite State Machine is crucial for managing G1's complex locomotive behaviors. It defines distinct states like 'Idle' or 'Walk' and the rules for transitioning between them based on various events. This structured approach makes G1's behavior predictable and easier to manage.

Diagram cue: Simple FSM diagram showing Idle , Walk , Stand states and transitions between them with event labels.

Slide 5: FSM State Examples: Idle, Walk, Stand

Section: Locomotive Architecture & FSM **Purpose:** Illustrate common FSM states with examples. **On-slide text:**

Common G1 FSM States in Action

- **Idle** : Robot is stationary, awaiting commands, minimal power consumption.
- **Walk** : Robot is actively moving, executing a gait pattern.
- **Stand** : Robot is stationary but maintaining an upright, balanced posture.

Speaker notes: Let's look at some practical examples of FSM states. Idle is when G1 is waiting, conserving energy. Walk is its active locomotion state, and Stand is a stable, upright pose. Understanding these states is key to commanding G1 effectively.

Diagram cue: Icons or simple illustrations representing G1 in Idle , Walk , and Stand postures.

Slide 6: High-Level Controller Interaction with FSM

Section: Locomotive Architecture & FSM **Purpose:** Explain how high-level commands trigger FSM state changes. **On-slide text:**

Commanding G1: High-Level to FSM Flow

- **User Command:** (e.g., "start walking")
- **High-Level Controller:** Translates command into FSM event.
- **FSM:** Processes event, initiates state transition (e.g., Idle to Walk).

Speaker notes: The high-level controller acts as an interpreter, taking user-friendly commands and converting them into events that the FSM understands. This allows for intuitive control while maintaining the FSM's structured state management. We'll see how a simple command like "start walking" can initiate a complex sequence of FSM state changes.

Diagram cue: Flowchart showing user command -> High-Level Controller -> FSM event -> FSM state transition.

Slide 7: Low-Level Motor Drivers

Section: Locomotive Architecture & FSM **Purpose:** Describe the role of low-level motor drivers in executing FSM commands. **On-slide text:**

The Muscles of G1: Low-Level Motor Drivers

- **Direct Actuator Control:** Translates FSM outputs into motor commands.
- **PID Control Loops:** Ensures precise joint position and velocity.
- **Sensor Feedback:** Uses encoder data for closed-loop control.

Speaker notes: At the lowest level, motor drivers are responsible for the physical execution of movement. They take commands from the FSM and translate them into precise electrical signals for the actuators, using PID control loops and sensor feedback to ensure accurate and stable motion. This is where the digital meets the physical.

Diagram cue: Diagram showing a motor driver receiving commands, controlling a motor, and receiving feedback from an encoder in a closed-loop system.

Slide 8: Velocity Control - Introduction

Section: Velocity, Balance & JSON-CLI **Purpose:** Introduce the concept of velocity control for G1 locomotion. **On-slide text:**

Setting the Pace: G1 Velocity Control

- **Goal:** Command G1 to move at a desired speed and direction.
- **Linear Velocity:** Forward/backward, sideways movement.
- **Angular Velocity:** Turning (rotation).

Speaker notes: Velocity control is how we tell G1 how fast and in what direction to move. We'll explore both linear velocity, which dictates its translational movement, and angular velocity,

which controls its turning. This is a fundamental aspect of dynamic locomotion.

Diagram cue: Arrows indicating linear (forward, backward, sideways) and angular (rotation) velocities on a G1 humanoid figure.

Slide 9: Implementing Linear Velocity Commands

Section: Velocity, Balance & JSON-CLI **Purpose:** Explain how to send linear velocity commands to G1. **On-slide text:**

Moving Straight: Sending Linear Velocity Commands

- **JSON Structure:** `{"linear_x": 0.5, "linear_y": 0.0}` (meters/second).
- **CLI Tool:** `g1_cmd_vel --linear_x 0.5 .`
- **Impact:** G1 moves forward at 0.5 m/s.

Speaker notes: To command G1's linear velocity, we'll use a simple JSON structure. For instance, setting `linear_x` to 0.5 will make G1 move forward at half a meter per second. We'll demonstrate how to send these commands using a command-line interface tool.

Diagram cue: Example JSON-CLI command and a G1 humanoid moving forward with a velocity vector.

Slide 10: Implementing Angular Velocity Commands

Section: Velocity, Balance & JSON-CLI **Purpose:** Explain how to send angular velocity commands to G1. **On-slide text:**

Turning G1: Sending Angular Velocity Commands

- **JSON Structure:** `{"angular_z": 0.2}` (radians/second).
- **CLI Tool:** `g1_cmd_vel --angular_z 0.2 .`
- **Impact:** G1 rotates counter-clockwise at 0.2 rad/s.

Speaker notes: Similarly, angular velocity commands control G1's rotation. A positive `angular_z` value will make G1 turn counter-clockwise. We'll practice sending these commands to achieve controlled turning movements.

Diagram cue: Example JSON-CLI command and a G1 humanoid rotating with an angular velocity vector.

Slide 11: Managing Balance States During Transitions

Section: Velocity, Balance & JSON-CLI **Purpose:** Discuss how G1 maintains balance when transitioning between movements. **On-slide text:**

Staying Upright: Balance Management in Transitions

- **Dynamic Balance:** Active control to prevent falling.
- **Center of Mass (CoM) Adjustment:** Shifting weight for stability.
- **Foot Placement Strategy:** Adapting steps to maintain equilibrium.

Speaker notes: Maintaining balance is a continuous process, especially during transitions between different movements like starting to walk or changing direction. G1 employs dynamic balance control, constantly adjusting its Center of Mass and adapting its foot placement to ensure it remains stable.

Diagram cue: Sequence of diagrams showing G1 shifting its CoM and adjusting foot placement during a transition from standing to walking.

Slide 12: Stance Configurations via JSON-CLI - Introduction

Section: Velocity, Balance & JSON-CLI **Purpose:** Introduce the concept of configuring G1's stance using JSON-CLI. **On-slide text:**

G1's Posture: Configuring Stance with JSON-CLI

- **Stance:** The robot's base posture (height, body orientation).
- **JSON-CLI:** Command-line interface using JSON for structured input.
- **Benefits:** Precise and repeatable posture adjustments.

Speaker notes: Stance configuration allows us to define G1's base posture, including its height and body orientation. We'll use JSON-CLI structures to send these configurations, enabling precise and repeatable adjustments to G1's starting position or resting pose.

Diagram cue: A G1 humanoid in a default stance, with arrows indicating adjustable parameters (height, pitch, roll).

Slide 13: Configuring Stance Height

Section: Velocity, Balance & JSON-CLI **Purpose:** Explain how to adjust G1's height using JSON-CLI. **On-slide text:**

Tall or Short: Adjusting G1's Stance Height

- **JSON Structure:** `{"stance_height": 0.8}` (meters).
- **CLI Tool:** `g1_set_stance --height 0.8`.
- **Impact:** G1 adjusts its overall height.

Speaker notes: We can command G1 to adjust its height, for example, to navigate under obstacles or reach higher objects. The `stance_height` parameter in our JSON-CLI structure allows for precise control over this crucial aspect of its posture.

Diagram cue: Two G1 humanoids, one taller and one shorter, illustrating the effect of `stance_height`.

Slide 14: Configuring Stance Pitch and Roll

Section: Velocity, Balance & JSON-CLI **Purpose:** Explain how to adjust G1's body orientation (pitch and roll) using JSON-CLI. **On-slide text:**

Leaning In: Adjusting G1's Stance Pitch and Roll

- **JSON Structure:** `{"stance_pitch": 0.1, "stance_roll": -0.05}` (radians).
- **CLI Tool:** `g1_set_stance --pitch 0.1 --roll -0.05`.
- **Impact:** G1 leans forward (pitch) and slightly to the right (roll).

Speaker notes: Beyond height, we can also control G1's body orientation using `stance_pitch` and `stance_roll`. Pitch controls forward/backward lean, while roll controls sideways lean. These parameters are vital for fine-tuning balance and preparing for specific tasks.

Diagram cue: A G1 humanoid illustrating positive pitch (leaning forward) and negative roll (leaning right).

Slide 15: JSON-CLI Structures for Stance Configuration

Section: Velocity, Balance & JSON-CLI **Purpose:** Provide a consolidated view of JSON-CLI structures for stance. **On-slide text:**

Unified Posture: JSON-CLI for Full Stance Control

- **Combined Command:** `g1_set_stance --config '{"height": 0.7, "pitch": 0.05, "roll": 0.0}'`.
- **Flexibility:** Adjust multiple parameters in a single command.
- **Readability:** JSON format is human-readable and machine-parseable.

Speaker notes: For comprehensive stance control, we can combine all parameters into a single JSON-CLI command. This allows for flexible and readable configuration of G1's posture, making it easy to set up specific initial conditions for experiments or tasks.

Diagram cue: A complete JSON-CLI command example for stance configuration, highlighting the key-value pairs.

Slide 16: Arm Control Infrastructure - Introduction

Section: Arm Control & SDK Merging **Purpose:** Introduce the arm control infrastructure in G1. **On-slide text:**

G1's Dexterity: Arm Control Infrastructure

- **Goal:** Enable G1 to manipulate objects and interact with its environment.
- **Dual Approach:** Low-level SDK for precise joint control, high-level APIs for complex actions.

Speaker notes: Now, let's shift our focus to G1's arms. The arm control infrastructure is designed to give G1 dexterity, allowing it to perform tasks like grasping, pushing, or gesturing. We'll explore a dual approach: precise low-level control and convenient high-level actions.

Diagram cue: A G1 humanoid with its arms highlighted, performing a simple manipulation task.

Slide 17: Low-Level Joint Configuration SDKs

Section: Arm Control & SDK Merging **Purpose:** Explain the role of low-level SDKs for direct joint control. **On-slide text:**

Fine-Grained Control: Low-Level Joint SDKs

- **Direct Motor Access:** Control individual joint angles, velocities, and torques.
- **High Precision:** Ideal for calibration, fine adjustments, and custom movements.
- **SDK Functions:** `set_joint_angle(joint_id, angle)` , `get_joint_torque(joint_id)` .

Speaker notes: The low-level Joint Configuration SDK provides direct access to G1's arm motors. This is where you can command individual joint angles, velocities, or even torques with high precision. It's essential for tasks requiring very specific movements or for advanced users developing new control algorithms.

Diagram cue: Diagram of a G1 arm with individual joints labeled, showing how each joint can be controlled independently.

Slide 18: High-Level Action APIs

Section: Arm Control & SDK Merging **Purpose:** Explain the role of high-level APIs for pre-defined arm movements. **On-slide text:**

Simplified Actions: High-Level Action APIs

- **Pre-defined Movements:** (e.g., `reach_for_object(object_id)` , `wave_hand()`).
- **Abstraction:** Hides complex joint kinematics and trajectory generation.
- **Ease of Use:** Faster development for common tasks.

Speaker notes: In contrast to low-level control, high-level Action APIs offer pre-defined, complex movements. These APIs abstract away the intricate details of joint kinematics, allowing you to command G1's arms with simple, intuitive functions like reaching for an object or waving. This significantly speeds up development for common tasks.

Diagram cue: Icons representing various high-level actions (e.g., a hand reaching, a hand waving).

Slide 19: Merging SDKs with Action APIs - The Bridge

Section: Arm Control & SDK Merging **Purpose:** Explain how low-level SDKs and high-level APIs are integrated. **On-slide text:**

Bridging the Gap: SDK and API Integration

- **Hybrid Approach:** Combine precision of SDK with convenience of APIs.
- **Action API Implementation:** Often built on top of SDK functions.
- **Flexibility:** Choose the right level of control for each task.

Speaker notes: The real power comes from merging these two approaches. High-level Action APIs are often implemented using the low-level SDK functions. This hybrid approach gives developers the flexibility to choose the most appropriate level of control for any given task, from broad gestures to minute adjustments.

Diagram cue: Diagram showing the Action API layer sitting on top of the SDK layer, which then interfaces with the hardware.

Slide 20: Understanding G1 Joint Mapping

Section: Arm Control & SDK Merging **Purpose:** Detail the joint mapping of G1's arms for precise control. **On-slide text:**

G1's Anatomy: Understanding Joint Mapping

- **Kinematic Chain:** The sequence of joints and links forming the arm.
- **Joint IDs:** Unique identifiers for each motor (e.g., `shoulder_pitch` , `elbow_roll`).
- **Degrees of Freedom (DoF):** The number of independent movements each arm can make.

Speaker notes: To effectively control G1's arms, we need to understand its joint mapping. This involves knowing the kinematic chain – the order of joints and links – and the unique IDs for each motor. We'll also discuss the concept of Degrees of Freedom, which defines the arm's overall movement capabilities.

Diagram cue: Detailed diagram of a G1 arm, with each joint clearly labeled with its ID and axis of rotation.

Slide 21: Coordinating Arm-Torso Movements

Section: Arm Control & SDK Merging **Purpose:** Explain the importance and methods of coordinating arm and torso movements. **On-slide text:**

Full Body Harmony: Coordinating Arm and Torso Movements

- **Enhanced Reach:** Torso movement extends arm workspace.
- **Improved Balance:** Torso adjustments maintain stability during arm motion.
- **Integrated Control:** Commands sent to both arm and torso controllers simultaneously.

Speaker notes: For many tasks, G1's arms don't operate in isolation. Coordinating arm movements with torso adjustments can significantly enhance reach, improve balance, and make interactions more natural. We'll look at how to send integrated commands to achieve this full-body harmony.

Diagram cue: A G1 humanoid reaching for an object, showing how the torso leans to assist the arm's extension.

Slide 22: SDK and API Integration - Practical Workflows

Section: Integration & Execution **Purpose:** Introduce practical workflows for integrating SDK and API commands. **On-slide text:**

Seamless Control: Practical Integration Workflows

- **Scenario-Based:** Choose SDK or API based on task complexity.
- **Hybrid Scripts:** Combine both for optimal control.
- **Example:** Use API for general grasp, then SDK for fine-tuning grip force.

Speaker notes: In real-world applications, you'll often use a combination of SDK and API commands. We'll explore practical workflows, demonstrating how to create hybrid scripts that leverage the strengths of both, for example, using an API for a general grasp and then the SDK for fine-tuning the grip force.

Diagram cue: Flowchart showing a decision tree for choosing between SDK and API, leading to a hybrid script example.

Slide 23: Sending JSON-Based Commands

Section: Integration & Execution **Purpose:** Detail the process of sending JSON-based commands to G1. **On-slide text:**

Your Instructions: Sending JSON-Based Commands

- **Standard Format:** JSON is widely used for structured data exchange.
- **Command Structure:** Key-value pairs for actions and parameters.
- **CLI or API Endpoint:** Methods for delivering commands to G1.

Speaker notes: JSON is our preferred format for sending structured commands to G1. We'll review the common command structures, including key-value pairs for actions and parameters, and discuss how to deliver these commands either through a command-line interface or directly to an API endpoint.

Diagram cue: Example of a JSON command for a G1 action, highlighting the structure.

Slide 24: Reading Joint Feedback for Arms

Section: Integration & Execution **Purpose:** Explain how to read and interpret joint feedback data from G1's arms. **On-slide text:**

G1's Response: Reading Arm Joint Feedback

- **Real-time Data:** Joint angles, velocities, and torques.
- **DDS Topics:** Feedback published on dedicated DDS topics.
- **Monitoring Tools:** Use `dds_msg_echo` or custom subscribers.

Speaker notes: To ensure our commands are executed correctly, we need to read feedback from G1's arm joints. This real-time data, including joint angles, velocities, and torques, is published on dedicated DDS topics. We'll learn how to monitor this feedback using existing tools or by creating custom subscribers.

Diagram cue: Screenshot or mock-up of terminal output showing real-time joint feedback data.

Slide 25: Validating Synchronized Locomotive-Arm Execution

Section: Integration & Execution **Purpose:** Emphasize the importance of validating coordinated movements between locomotion and arms. **On-slide text:**

Perfect Harmony: Validating Synchronized Execution

- **Integrated Test Cases:** Design tests that involve both locomotion and arm tasks.
- **Timing Analysis:** Ensure actions occur in the correct sequence and within tolerances.

- **Visual Inspection & Data Logging:** Observe behavior and analyze recorded data.

Speaker notes: For complex tasks, G1's locomotion and arm movements must be perfectly synchronized. We'll discuss how to design integrated test cases, perform timing analysis to ensure correct sequencing, and use both visual inspection and data logging to validate synchronized execution.

Diagram cue: Timeline diagram showing synchronized events for locomotion and arm movements.

Slide 26: Control Discipline - Best Practices

Section: Integration & Execution **Purpose:** Introduce best practices for maintaining control discipline in G1 operations. **On-slide text:**

Mastering G1: Cultivating Control Discipline

- **Clear Intent:** Define precise goals for each command.
- **Incremental Testing:** Test small changes before integrating larger ones.
- **Parameter Validation:** Ensure commands are within safe operating limits.
- **Emergency Stop Readiness:** Always be prepared to halt operation.

Speaker notes: Control discipline is about developing good habits for interacting with G1. This includes having clear intent for your commands, testing incrementally, validating parameters, and always being ready to initiate an emergency stop. These practices minimize errors and enhance safety.

Diagram cue: A checklist of control discipline best practices.

Slide 27: Joint Validation - Ensuring Accuracy

Section: Integration & Execution **Purpose:** Explain methods for validating the accuracy of joint movements. **On-slide text:**

Precision in Motion: Joint Validation Techniques

- **Position Verification:** Compare commanded vs. actual joint angles.
- **Torque Monitoring:** Detect unexpected forces or collisions.
- **Range of Motion Checks:** Ensure joints operate within physical limits.
- **Repeatability Tests:** Verify consistent movement over multiple executions.

Speaker notes: Joint validation is crucial for ensuring G1's arms move accurately and safely. We'll cover techniques like comparing commanded versus actual joint positions, monitoring torques for anomalies, checking ranges of motion, and performing repeatability tests to ensure consistent performance.

Diagram cue: Graph comparing commanded vs. actual joint angle over time.

Slide 28: Command Latency Awareness

Section: Integration & Execution **Purpose:** Discuss the impact of command latency and how to manage it. **On-slide text:**

Time is Critical: Understanding Command Latency

- **Definition:** The delay between sending a command and G1's response.
- **Sources:** Network delays, processing time, actuator response.
- **Mitigation:** Optimize communication, use real-time operating systems.
- **Impact:** Affects responsiveness and synchronization.

Speaker notes: Command latency, the delay between sending a command and G1's response, can significantly impact performance and synchronization. We'll explore the sources of latency, discuss mitigation strategies like optimizing communication, and understand its critical impact on real-time control.

Diagram cue: Timeline diagram illustrating command transmission, processing, and execution with associated delays.

Slide 29: Day 6 Wrap-Up

Section: Wrap-Up & Review **Purpose:** Conclude the Day 6 session and reinforce key learning points. **On-slide text:**

Day 6: Control Achieved!

- **Locomotive Control:** Architecture, FSM, motor drivers.
- **Velocity & Balance:** Commands, stance configuration.
- **Arm Control:** SDKs, Action APIs, joint mapping.
- **Integration:** JSON commands, feedback, synchronized execution.

Speaker notes: Congratulations! You've successfully navigated Day 6, gaining crucial insights into G1 humanoid locomotion and arm control. You now have a solid foundation to command G1's movements and interactions with precision and safety.

Diagram cue: A celebratory graphic or a summary infographic of the day's topics.

Slide 30: Next Steps and Resources

Section: Wrap-Up & Review **Purpose:** Provide guidance for further learning and available resources. **On-slide text:**

Your Journey Continues: Next Steps

- **Practice Exercises:** Apply what you've learned in hands-on labs.
- **G1 Documentation:** Dive deeper into specific control modules.
- **Community Forums:** Engage with other G1 developers.
- **Day 7 Preview:** What's next in your G1 learning path?

Speaker notes: Your learning doesn't stop here. I encourage you to engage with the practice exercises, explore the comprehensive G1 documentation, and join the community forums. And get ready for Day 7, where we'll build upon today's knowledge!

Diagram cue: Icons representing practice, documentation, community, and a forward arrow for next steps.

Slide 31: References

Section: References **Purpose:** Provide a list of resources for further learning. **On-slide text:**

Further Reading and Resources

- [Robotics Systems Architectures](#)
- [Finite State Machines in Robotics](#)
- [JSON Standard](#)

Speaker notes: Here are some recommended resources for those who wish to delve deeper into robotics system architectures, finite state machines, and the JSON standard. These will provide more in-depth technical details and advanced concepts.

Diagram cue: Icons representing books, websites, and documentation.

Slide 32: Day 6 Outcome Coverage Report

Section: Wrap-Up & Review **Purpose:** Summarize the coverage of required Day 6 outcomes.
On-slide text:

Comprehensive Coverage: Day 6 Outcomes Achieved

Outcome	Coverage (%)	Notes
Locomotive Control Subsystems	100%	System architecture, FSM state management, high-level to low-level interaction.
Velocity Control and Balance States	100%	Velocity commands (linear/angular), balance states, stance configuration via JSON-CLI.
Arm Control Infrastructure	100%	Merging low-level SDKs with high-level Action APIs, joint mapping, arm-torso coordination.
SDK and API Integration	100%	Practical workflows, JSON commands, joint feedback, synchronized execution.

Speaker notes: We have thoroughly covered all required Day 6 outcomes, ensuring a complete understanding of G1 humanoid locomotion and arm control. Each topic has been addressed with beginner-friendly explanations and practical considerations.

Diagram cue: A pie chart or bar graph showing 100% coverage for each outcome.

Slide 33: Copyright and Disclaimer

Section: Copyright **Purpose:** State copyright information and disclaimers. **On-slide text:**

© 2026 Vinci AI. All rights reserved.

Property of Vinci AI — Do Not Distribute

Disclaimer: This material is for educational purposes only and should not be used for actual robot operation without proper training and supervision.

Speaker notes: This concludes our Day 6 session. Please remember that this material is the property of Vinci AI and is intended for educational use only. Always prioritize safety and follow proper protocols when working with robotics.

Diagram cue: Vinci AI logo, subtle diagonal watermark, copyright tag.